

Slack Legal OS — Setup Guide

This guide takes you from a fresh clone to a working AI legal assistant running in **your own** Slack workspace, connected to **your own** Clio account.

What you'll end up with: a Slack bot you can `@mention` (or DM, or trigger with `/legal-os`) that looks up Clio matters/contacts/bills, renders charts inline, and answers questions with web search — plus a private admin web UI.

 **Time:** ~45–60 minutes the first time.  **Cost:** Can be done on free tiers to start. A Vercel **Pro** plan is recommended (and required later for Sandbox + Cron features on the roadmap).

0. Accounts you'll need

Create these first (all have free tiers to start). Keep a scratch doc open to paste keys into as you go.

Service	What it's for	Sign up
GitHub	Host your fork of the code	https://github.com
Vercel	Hosting + database/blob marketplace + AI Gateway	https://vercel.com/signup
Neon	Postgres database (installed <i>through</i> Vercel)	provisioned in Step 3
Clerk	Admin-UI login	https://dashboard.clerk.com
Slack	The workspace where the bot lives (you need admin rights to install apps)	your own workspace
Clio	The case-management system the bot reads/writes	https://app.clio.com (+ developer access)
Perplexity	Web-search tool for the agent	https://www.perplexity.ai

Local tools: [Node.js 24+](#), [pnpm](#) (`npm i -g pnpm`), [git](#), and the [Vercel CLI](#) (`npm i -g vercel`).

1. Get the code

```
git clone https://github.com/YASAM1/slack-legal-os.git
cd slack-legal-os
pnpm install
cp .env.example .env.local
```

You'll fill in `.env.local` as you collect keys below. Every variable is documented inline in `.env.example`.

2. Create the Vercel project (do this early — it gives you your URL)

Several services (Slack, Clio) need to know your app's public URL, so create the Vercel project **first**.

```
vercel login
vercel link      # choose "create a new project", name it e.g. slack-legal-os
```

Your production URL will be `https://<project-name>.vercel.app` (Vercel shows it after linking). Write it down — this guide calls it **YOUR-APP-URL**.

 You can also create the project from the Vercel dashboard by importing your GitHub repo. Either way, note the production URL.

3. Provision the database (Neon, via Vercel)

1. In the Vercel dashboard, open your project → **Storage** → **Create Database** → **Neon** (Postgres) → follow the prompts.
2. This auto-injects `DATABASE_URL` **and** `DATABASE_URL_UNPOOLED` into your Vercel project's environment variables.
3. While you're in Storage, also create a **Blob** store (→ **Create** → **Blob**). This injects `BLOB_READ_WRITE_TOKEN`, used for chart image uploads.

 `DATABASE_URL` is the pooled connection (used at runtime). `DATABASE_URL_UNPOOLED` is the direct connection used to run migrations. The Neon integration gives you both.

4. Set up Clerk (admin login)

1. Go to <https://dashboard.clerk.com> → **Create application**. Enable **Email** (and Google if you like) as a sign-in method.
2. From **API Keys**, copy:
 - `NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY` (starts with `pk_`)
 - `CLERK_SECRET_KEY` (starts with `sk_`)
3. Paste both into `.env.local`.
4. Set **your own email** as the admin allowlist in `.env.local`:

```
ADMIN_ALLOWED_EMAILS=you@yourfirm.com
```

(Comma-separate multiple admins.) Anyone signing in whose email isn't on this list gets a 403.

The app ships its own sign-in/sign-up pages at `/sign-in` and `/sign-up` — no extra Clerk config needed; the default `.env` values already point at them.

5. Generate the encryption key

The Clio refresh token is encrypted at rest (AES-256-GCM). Generate a 32-byte key:

```
openssl rand -hex 32
```

Paste the 64-character result into `.env.local`:

```
ENCRYPTION_KEY=<paste the 64 hex chars here>
```

6. Get an AI Gateway key (the agent's brain)

The agent talks to Claude through the **Vercel AI Gateway**.

1. Vercel dashboard → **AI Gateway** → create an **API key**.
2. Put it in `.env.local`:

```
AI_GATEWAY_API_KEY=<your key>
```

✅ **On Vercel (production), you can leave this blank** — the deployment authenticates to the Gateway automatically via OIDC.
You only *need* a key for **local development**.

The default models are already set in `.env.example`:

```
AGENT_MODEL=anthropic/claude-sonnet-4.6  
EMBEDDING_MODEL=openai/text-embedding-3-small # used by the roadmap KB feature
```

7. Get a Perplexity API key (web search)

1. <https://www.perplexity.ai> → Settings → **API** → generate a key.
2. Add to `.env.local`:

```
PERPLEXITY_API_KEY=<your key>
```

8. Create the Slack app

The repo includes an importable manifest at `slack-manifest.json`.

1. **Swap in your URL.** Open `slack-manifest.json` and replace every `https://YOUR-APP.vercel.app/api/webhooks/slack` with `https://YOUR-APP-URL/api/webhooks/slack`. (One endpoint handles events, the slash command, *and* interactivity.)
2. Go to <https://api.slack.com/apps> → **Create New App** → **From an app manifest** → pick your workspace → paste the edited JSON.
3. **Install to Workspace** (under *Install App*). Approve the requested scopes.
4. Copy the **Bot User OAuth Token** (`xoxb-...`) → `.env.local` as `SLACK_BOT_TOKEN`.
5. Under **Basic Information** → **App Credentials**, copy the **Signing Secret** → `.env.local` as `SLACK_SIGNING_SECRET`.

The manifest already requests the right scopes (`app_mentions:read` , `chat:write` , `commands` , `files:write` , `im:history` , `users:read` , etc.) and subscribes to the right events.

⚠ Slack will try to verify your **Request URL** when you save the manifest. That URL must already be deployed and responding (see Step 11). If verification fails the first time, finish the deploy, then come back to **Event Subscriptions** and click **Retry**.

9. Create the Clio developer app

1. Log in to Clio → **Settings** → **Developers** (or visit your region's developer portal) → **New Application**.
2. Set the **Redirect URI** to exactly:

```
https://YOUR-APP-URL/api/clio/auth/callback
```

3. Request the scopes your firm needs (read/write to matters, contacts, activities, bills, users).
4. Copy the **Client ID** and **Client Secret** → `.env.local`:

```
CLIO_CLIENT_ID=<...>
CLIO_CLIENT_SECRET=<...>
CLIO_REDIRECT_URI=https://YOUR-APP-URL/api/clio/auth/callback
CLIO_BASE_URL=https://app.clio.com          # use https://eu.app.clio.com if your firm is on Clio EU
```

10. Push all env vars to Vercel

Your keys are in `.env.local`. The fastest way to get them into Vercel:

```
# Add each variable to Production (and Preview/Development as desired).
# Easiest path: open the Vercel dashboard → Project → Settings → Environment Variables
# and paste each key/value. DATABASE_URL*, BLOB_READ_WRITE_TOKEN are already there from Steps 3.
```

Make sure these exist in Vercel **Production** env:

```
ENCRYPTION_KEY , NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY , CLERK_SECRET_KEY , ADMIN_ALLOWED_EMAILS , SLACK_BOT_TOKEN ,
SLACK_SIGNING_SECRET , CLIO_CLIENT_ID , CLIO_CLIENT_SECRET , CLIO_REDIRECT_URI , CLIO_BASE_URL , PERPLEXITY_API_KEY .
```

(`AI_GATEWAY_API_KEY` is optional in production thanks to OIDC. `CRON_SECRET` , `VERCEL_OIDC_TOKEN` are for roadmap features — skip for now.)

💡 Already added them in the dashboard? Pull them back to your machine for local dev with `vercel env pull .env.local`.

11. Deploy

⚠️ **Known build quirk — read this.** The standard Vercel hosted (Git-connected) build currently fails on this project with an opaque `TypeError: The "path" argument must be of type string`. The reliable workaround is to build locally and upload the prebuilt output:

```
pnpm deploy:prebuilt
```

This runs `vercel build --prod` on your machine and uploads it with `vercel deploy --prebuilt --prod`. (Root cause is still under investigation; see Troubleshooting.)

After it deploys, visit `https://YOUR-APP-URL` — you should get the app. Visit `https://YOUR-APP-URL/admin` and you'll be asked to sign in via Clerk; sign in with an allowlisted email.

Now go back to **Step 8** and make sure Slack successfully verified the Request URL (Event Subscriptions should show a green ✓). Retry if needed.

12. Run database migrations

This enables the `pgvector` extension and creates all tables in a dedicated `legal_os` Postgres schema. Run it once against your Neon DB (it reads `DATABASE_URL_UNPOOLED` from `.env.local`):

```
vercel env pull .env.local    # if you haven't already, get the DB URLs locally
pnpm db:migrate
```

You should see:

```
Enabling pgvector extension...
Running migrations...
Migrations complete.
```

(Optional) inspect the DB with `pnpm db:studio`.

13. Connect Clio

1. Open `https://YOUR-APP-URL/admin/integrations`.
2. Click **Connect Clio** → you'll be sent to Clio to authorize → on success you're redirected back showing **Connected**.

3. Behind the scenes, the refresh token is encrypted and stored; the client auto-refreshes access tokens and retries on 401.

14. Invite the bot and test it

1. In Slack, invite the bot to a channel: `/invite @Legal OS` (or whatever you named it).

2. Try these:

- o `@Legal OS hello` → it replies in-thread.
- o `@Legal OS list the matters opened this month` → it calls Clio.
- o `@Legal OS show me matters opened this month as a bar chart by practice area` → it renders a chart and posts the image inline. 📊
- o `/legal-os what's a recent ruling on <topic>?` → it uses web search and cites sources.

3. Any Clio **write** (create/update) will ask you to confirm in-thread before proceeding.

That's a working install. ✅

Local development (optional)

```
vercel env pull .env.local # sync env from Vercel
pnpm dev                  # http://localhost:3000
```

For Slack to reach your local machine, point the Slack app's Request URL at a tunnel (e.g. `ngrok http 3000` → use the https URL + `/api/webhooks/slack`). Remember to switch it back to your Vercel URL afterward.

Environment variable reference

Variable	Required now?	Where it comes from
DATABASE_URL / DATABASE_URL_UNPOOLED	✓	Neon integration (Step 3)
ENCRYPTION_KEY	✓	openssl rand -hex 32 (Step 5)
NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY / CLERK_SECRET_KEY	✓	Clerk (Step 4)
ADMIN_ALLOWED_EMAILS	✓	you choose (Step 4)
SLACK_BOT_TOKEN / SLACK_SIGNING_SECRET	✓	Slack app (Step 8)
CLIO_CLIENT_ID / CLIO_CLIENT_SECRET / CLIO_REDIRECT_URI / CLIO_BASE_URL	✓	Clio app (Step 9)
PERPLEXITY_API_KEY	✓	Perplexity (Step 7)
BLOB_READ_WRITE_TOKEN	✓	Vercel Blob (Step 3)
AI_GATEWAY_API_KEY	local only	Vercel AI Gateway (Step 6) — auto via OIDC in prod
AGENT_MODEL / EMBEDDING_MODEL / CODEGEN_MODEL	defaults set	.env.example
CRON_SECRET / VERCEL_OIDC_TOKEN	roadmap	Vercel (future Cron/Sandbox features)

Troubleshooting

Symptom	Fix
Vercel hosted build fails with TypeError: The "path" argument must be of type string	Use pnpm deploy:prebuilt (builds locally, uploads prebuilt). Known issue with this Next 16 + dependency mix.
Slack "Your URL didn't respond / verification failed"	Make sure the app is deployed and live first, then Retry in Event Subscriptions. Confirm the URL is exactly https://YOUR-APP-URL/api/webhooks/slack .
Bot doesn't reply	Check SLACK_SIGNING_SECRET and SLACK_BOT_TOKEN are set in Vercel Production; confirm the bot was invited to the channel; check the function logs in Vercel.
403 Forbidden on /admin	Your Clerk email isn't in ADMIN_ALLOWED_EMAILS . Add it (in Vercel env) and redeploy.
ENCRYPTION_KEY must be 32 bytes	The key must be 64 hex chars. Regenerate with openssl rand -hex 32 .
Clio connect fails / redirect mismatch	The Clio app's Redirect URI must match CLIO_REDIRECT_URI exactly, including https:// and no trailing slash. EU firms: set CLIO_BASE_URL=https://eu.app.clio.com .
Migrations can't connect	Ensure DATABASE_URL_UNPOOLED is present locally (vercel env pull .env.local).
Charts don't post	Confirm the Blob store exists and BLOB_READ_WRITE_TOKEN is set; check files:write scope is granted to the Slack app.

What's on the roadmap (not yet functional)

The admin UI has placeholder pages for these; the features are in progress:

- **Knowledge base** — curated Markdown docs + ingested Clio documents embedded into pgvector for cited retrieval.
- **Capability codegen** — describe a task in plain English → preview generated code → dry-run in Vercel Sandbox → activate.
- **Scheduling & daily report** — cron-driven capabilities and an automatic daily status report to a channel.
- **Guardrails & polish** — token spend caps, run-history viewers, alerting, tests.

You don't need `CRON_SECRET` or `VERCEL_OIDC_TOKEN` /Sandbox configured to use everything in this guide.